Bağlama Kavramı ve İsim kapsamları

9 Mart 2026 Pazartesi 10:08

Bağlama Kavramı

- Bir özellik ile programlama elemanının bağlanması
  - İsim
  - Adres,
  - Değer
  - vb
  - Değişken
  - altprogram
  - vb.

bellekte bulunmalı yoksa erişemeyiz

Bağlama Zamanı

- Dilin tasarımı sırasında
- Dilin gerçeklenmesi sırasında
- Dilin derlenmesi sırasında
- Dilin yüklenmesi veya çalışması sırasında

int a = 100; yazdığımızda int'e bağlanması derlenmesi sırasında olur.
100 değerinin atanması çalışma zamanında olur.
NOT: var kelimesi de derlenme zamanında örnektir.
List<T> gibi generic sınıflarda türü çalışma anında belirlenir
yorumlayıcıyla çalışan dillerde çalışma zamanında olur. (dinamik tip bağlama)

Statik Tip Bağlama

- Örtülü tip bağlama
- Dışsal tip bağlama

Perl dilinde örnek kod

```
$var1 = 9;
$var2 = 'string';
@meyveler=("elmalar","armutlar","kavunlar");
$meyveler[2]
```

Java dilinde dışsal tip bağlama

```
public static void main(String[] args) {
    String x;
    x="Mustafa";
}
```

örtülü

STATİK: Çalışmadan önce türü belli

derleyicili diller genelde dışsalı kullanır
örtülü dillerde genelde örtülü kullanır

Dinamik Tip Bağlama

- Bir değişkenin tipi çalışma zamanında ve değişkenin bağlandığı değer ile belirleniyorsa, bu dil dinamik tip bağlamalı bir dildir.

Java dilinde kalıtım kullanılarak dinamik tip bağlamanın sağlanması

```
public class Canli {
   public void Kos(){
      System.out.println("Canlı Koşuyor.");
   }
}
public class Insan extends Canli {
   @Override
   public void Kos(){
      System.out.println("Insan Koşuyor.");
   }
}
public class Hayvan extends Canli {
   @Override
   public void Kos(){
      System.out.println("Hayvan Koşuyor.");
   }
}
public static void main(String[] args) {
      Canli c = new Insan();
      c.Kos();
      c=new Hayvan();
      c.Kos();
   }
```

Genelde yorumlayıcılı dillerde olur. fakat aslında ndp de benzer olarak çalışır. Çalışma zamanına kadar kimin kimden miras aldığı belli olmaz.

örnek:
İf(Canli c){ ...
şeklinde bir kod var buraya Insan sınıfından nesne de gönderilebilir fakat derleyici bu kontrolü yapamaz. Burdaki tip türü değer atanınca belli olur.
veya
Canli c = new Insan() derken.

Bellek Bağlama

- Bir değişkenin erişilebilir bir bellek hücresi ile ilişkilendirilmesine bellek bağlama ya da bellek yeri ataması (memory allocation) denir.

heap bölgesi: dinamik yığıt şeklinde de adlandırılabilir.

fonksiyonlar her zaman yığıtta çağrılır.

Soldaki hücrelerin yaşam süreleri farklıdır.
Bellekte olan bir şeye erişebileceğin anlamına gelmez.
Kapsam bağlama olmazsa erişemezsin.

Statik bellek bölgesi	Çalışma Anı Yığını (RTS)	Heap
• global değişkenler (javada yoktur) (yaşam süresi programla aynı) (pi sayısı gibi şeyler global tanımlanır) • Statik lokal değişkenler (yaşam süresi aynı fakat kapsamı daha dar) (Nesneye dayalı da dahi statik olan elemanı burada oluşturulur)	**local bölge kapanınca otomatik silinir.** • local değişkenler • parametreler	**sen yok etmediğin sürece silinmez.** **buradaki değişkenler isimsizdir.** **Javada nesneler sadece burada oluşturulabilir.** **c++'da pointer ile burada oluşturulabilir.**

C++ için:
Ogrenci ogr; //stack
Ogrenci* ogr = new Ogrenci(); //heap
//iki ogr de stackte tutulur fakat ikincisi nesneyi heapte oluşturur.

Java için: (C#'ta da böyle)
Ogrenci ogr; //nesne oluşturmaz. sadece stackte onu tutucak bir referans oluştururs.
Ogrenci ogr = new Ogrenci(); //heapte nesne oluşturur. (ogr stack nesne heap.)

Çöp Toplayıcı:
Java c# python gibi dillerde sürekli programa ayrılmış ram bölgesini tarayıp heapteki çöpleri (referanssız kalmışları) otomatik temizler. Yavaşlık getirir.

C ve C++'da malloc mantığı:
içine yazılan değer kadar bellekte yer açar ve açılan yeri void* tipinde return eder.
Türlerin kapladığı alanlar makineden makineye değişeceğinden sizeof operatörü kullanılır

malloc açtığı yer hücresini void* döndürdüğü için sonrasında type casting yapılması gerekir:
int a = (int*) malloc(sizeof(int));
Ogrenci* ogr = (Ogrenci*)malloc(sizeof(Ogrenci));

Farklı Dillerin Bellek Kullanımı

Prog. Dilleri	Statik	Çalışma Anı Yığıtı	Heap
Ada		Lokal değişkenler Alt program parametreleri	Lokal değişkenler new (çöp toplayıcı)
C	Global değişkenler Statik lokal değişkenler	Lokal değişkenler Alt program parametreleri	malloc (free)
C++	C ile aynı Statik sınıf üyeleri	C ile aynı	new (delete)
Java	Statik Sınıf elemanları	Sadece ilkel tipli lokal değişkenler	Tüm nesneler (Çöp toplayıcı)
Fortran 77	Global değişkenler Lokal değişkenler Alt program parametreleri	Lokal değişkenler	
Pascal	Global değişkenler (derleyici bağımlı)	Global değişkenler (derleyici bağımsız) Lokal değişkenler Alt program parametreleri	new (dispose)

Kapsam Bağlama

- **Statik Kapsam Bağlama**
  Değişkenlerin kapsam alanları programın metinsel düzenine göre yapılır.
- **Dinamik Kapsam Bağlama**
  Bir ismin kapsamı, altprogramların çağrılış sırasına göre çalışma zamanında belirlenir.

c ve c++da

```
int a;
{
    int a;
    {
        int a;
    }
}
```

şeklinde yazılabilir.
kapsamları (scope) farklı olacağından 3 adet a tanımlanabilir

Programlama dilleri ya statik ya da dinamik bağlama kullanır. İkisi aynı anda kullanılmaz
statikte derleme anında hata varsa hata verir
dinamikte genelde hata varsa çalışma anında patlar.

Pascal dilinde statik kapsam bağlama:
(beginden başlar)

Statik Kapsam Bağlama

```
program L;
    var n: char ; {n, L' de bildirilmiş }
    procedure W;
    begin
        writeln(n) {W de n 'ye başvuru var.}
    end;
    procedure D;
        var n: char; {D de n tekrar bildirilmiş}
        begin
            n:= "D" ;
            W; { D' deki W' i çağırdı}
        end;
begin {L}
  n:= "L";
  W; {Ana program L' den W' u çağrıldı}
  D;
end.
```

1. Statik Kapsam Bağlama (Static / Lexical Scoping)

Statik kapsamda, bir değişkenin hangi değere sahip olacağı **kodun yazıldığı fiziksel yapıya (metne)** bakılarak derleme zamanında belirlenir. Derleyici, iç içe geçmiş bloklara (kapsamlara) dışa doğru bakar.

- **Özelliği:** Kodu okuyan bir insan için çok daha öngörülebilirdir. Çalışma zamanı (runtime) durumundan bağımsızdır.
- **Kodun Davranışı:**
  1. Ana program `W`'yi çağırır. `W` prosedürü içinde `writeln(n)` vardır.
  2. `W` kendi içine bakar, `n` diye yerel bir değişken bulamaz. Hemen kodda yazıldığı yerin bir üst kapsamına, yani global `program L`'ye bakar. Oradaki `n`'nin değeri `"L"`'dir. Ekrana **L** yazar.
  3. Ana program `D`'yi çağırır. `D` prosedürü kendi içinde yerel bir `n` değişkeni tanımlar ve değerini `"D"` yapar. Sonra `W`'yi çağırır.
  4. `W` prosedürü tekrar çalışır. `W`'yi kimin çağırdığının **hiçbir önemi yoktur**. `W` yine yazıldığı yere bakar, kendi içinde `n` bulamaz, globaldeki `n`'ye gider. Global `n` hala `"L"`'dir. Ekrana tekrar **L** yazar.
- **Statik Kapsam Çıktısı:**
  `L`
  `L`

L L yazar ekrana.

Dinamik Kapsam Bağlama

```
program L;
    var n: char ; {n, L' de bildirilmiş }
    procedure W;
    begin
        writeln(n) {W de n 'ye başvuru var.}
    end;
    procedure D;
        var n: char; {D de n tekrar bildirilmiş}
        begin
            n:= "D" ;
            W; { D' deki W' i çağırdı}
        end;
begin {L}
   n:= "L";
   W; {Ana program L' den W' u çağrıldı}
   D;
end.
```

2. Dinamik Kapsam Bağlama (Dynamic Scoping)

Dinamik kapsamda, değişkenin çözümlenmesi **çağrı yığınına (call stack)**, yani programın o anki çalışma sırasına göre çalışma zamanında (run-time) yapılır. Fonksiyon kendi içinde değişkeni bulamazsa, **kendisini çağıran fonksiyona** bakar.

- **Özelliği:** Davranışı önceden kestirmek çok zordur çünkü bir fonksiyonun içindeki değişken, fonksiyonun nereden çağrıldığına göre sürekli renk değiştirir. Bu da hata ayıklamayı (debugging) zorlaştırır.
- **Kodun Davranışı:**
  1. Ana program `W`'yi çağırır. `W` kendi içinde `n` bulamaz. "Beni kim çağırdı?" der. Ana program (`L`) çağırdığı için, ana programdaki `n` değerini kullanır. Ekrana **L** yazar.
  2. Ana program `D`'yi çağırır. `D` kendi yerel `n`'sini `"D"` yapar.
  3. `D`, `W`'yi çağırır. `W` prosedürü çalışır, kendi içinde `n` bulamaz. Yine "Beni kim çağırdı?" diye sorar. Bu sefer onu çağıran `D` prosedürüdür. Bu yüzden `D`'nin belleğindeki yerel `n` değerini (`"D"`) alır. Ekrana **D** yazar.
- **Dinamik Kapsam Çıktısı:**
  `L`
  `D`

L D yazar ekrana

C kodu örneği:

```
int x;
int main() {
  x = 2;
  f();
  g();
}
void f() {
  int x = 3;
  h();
}
void g() {
   int x = 4;
   h();
}
void h() {
   printf("%d\n",x); }
```

1. Statik Kapsam Bağlama (Static / Lexical Scoping)

Gerçek dünyada C, C++ ve Java gibi diller tam olarak böyle çalışır. Derleyici, kodun yazıldığı metin yapısına bakar.

- **Nasıl Davranır:** `h()` fonksiyonu derlenirken içine bakılır; `x` tanımlanmamış. Derleyici hemen kodun metin olarak bir üst kapsamına, yani **global kapsama** bakar. `h()` fonksiyonu metin olarak nerede duruyorsa, sadece oranın üstündeki kapsamlara erişebilir. Bu yüzden `h()` **her zaman** global `x`'i okuyacak şekilde derlenir. Onu kimin çağırdığının hiçbir önemi yoktur.
- **Adım Adım İzleme:**
  1. `main` çalışır, global `x` = 2 olur.
  2. `main`, `f()`'yi çağırır. `f()` kendi yerel `x`'ini 3 yapar (bu globali etkilemez).
  3. `f()`, `h()`'yi çağırır. `h()` çalışır ve her zaman ayarlandığı gibi global `x`'e bakar. Global `x` hala 2'dir. Ekrana **2** yazar.
  4. `main`, `g()`'yi çağırır. `g()` kendi yerel `x`'ini 4 yapar.
  5. `g()`, `h()`'yi çağırır. `h()` çalışır, yine global `x`'e bakar. Global `x` hala 2'dir. Ekrana tekrar **2** yazar.
- **Çıktı:**
  `2`
  `2`

2. Dinamik Kapsam Bağlama (Dynamic Scoping)

Eğer bu kod dinamik kapsamlı bir dilde (örneğin eski Lisp türevleri veya Bash betikleri) çalışıyor olsaydı, işler çağrı sırasına (call stack) göre değişirdi.

- **Nasıl Davranır:** `h()` fonksiyonu çalıştırıldığında kendi içinde `x`'i bulamazsa, metin yapısına bakmak yerine "Beni az önce kim çağırdıysa, gidip onun bellek alanındaki `x`'e bakayım" der. Yani geriye dönük bir iz sürme işlemi (call stack araması) yapar.
- **Adım Adım İzleme:**
  1. `main` çalışır, global `x` = 2 olur.
  2. `main`, `f()`'yi çağırır. `f()` kendi yerel `x`'ini 3 yapar.
  3. `f()`, `h()`'yi çağırır. `h()` kendi içinde `x` bulamaz. Geriye döner ve kendisini çağıran `f()` fonksiyonunun belleğine bakar. Orada `x=3` vardır. Ekrana **3** yazar.
  4. Program `main`'e döner, sonra `main`, `g()`'yi çağırır. `g()` kendi yerel `x`'ini 4 yapar.
  5. `g()`, `h()`'yi çağırır. `h()` kendi içinde `x` bulamaz. Geriye döner ve bu sefer kendisini çağıran `g()` fonksiyonunun belleğine bakar. Orada `x=4` vardır. Ekrana **4** yazar.
- **Çıktı:**
  `3`
  `4`

Bloklar

- Blok, herhangi bir bildirimin, bağlamanın kapsamının sınırlandırıldığı program bölgesidir.

```
int main(){
        char *str="Anlamaz";
        {
                char *str="Laftan";
                {
                        char *str="Murat";
                        printf("%s ",str);
                }
                printf("%s ",str);
        }
        printf("%s ",str);
        return 0;
}
```

stack (yığıt mantığıyla çalışır)